

Practice Assignment

Transactions · Joins · Indexing · Query Optimization

PART 1

```
CREATE database FoodDelivieri;
USE FoodDelivieri;
CREATE table customers(
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100),
  email VARCHAR(100) UNIQUE,
  signup_date DATE
);
CREATE TABLE restaurants (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100),
  cuisine VARCHAR(50)
);
CREATE TABLE orders (
  id INT PRIMARY KEY AUTO_INCREMENT,
  customer_id INT,
  restaurant_id INT,
  total DECIMAL(10,2),
  status VARCHAR(30),
  created_at DATETIME,

  FOREIGN KEY(customer_id) REFERENCES customers(id),
  FOREIGN KEY(restaurant_id) REFERENCES restaurants(id)
);
CREATE TABLE accounts (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50),
  balance DECIMAL(10,2)
);

INSERT INTO customers(name, email, signup_date) VALUES
('Aarav Patel','aarav@gmail.com','2025-01-05'),
('Riya Shah','riya@gmail.com','2025-01-08'),
('Vivaan Mehta','vivaan@gmail.com','2025-01-12'),
('Ananya Joshi','ananya@gmail.com','2025-01-18'),
('Dhruv Patel','dhruv@gmail.com','2025-01-25'),
('Priya Desai','priya@gmail.com','2025-02-01'),
('Yash Trivedi','yash@gmail.com','2025-02-06'),
('Neha Sharma','neha@gmail.com','2025-02-12'),
('Harsh Patel','harsh@gmail.com','2025-02-20'),
('Krishna Shah','krishna@gmail.com','2025-02-25'),
```

```
('Sneha Patel','sneha@gmail.com','2025-03-01'),
('Rahul Verma','rahul@gmail.com','2025-03-06'),
('Pooja Singh','pooja@gmail.com','2025-03-10'),
('Karan Joshi','karan@gmail.com','2025-03-15'),
('Mihir Shah','mihir@gmail.com','2025-03-20'),
('Nidhi Patel','nidhi@gmail.com','2025-03-25'),
('Jay Mehta','jay@gmail.com','2025-04-02'),
('Ishita Shah','ishita@gmail.com','2025-04-07'),
('Rohan Patel','rohan@gmail.com','2025-04-15'),
('Aditi Desai','aditi@gmail.com','2025-04-20');
INSERT INTO restaurants (name,cuisine) VALUES
('Pizza Hub','Italian'),
('Burger Point','Fast Food'),
('Spice Villa','Indian'),
('Dragon Wok','Chinese'),
('Taco Fiesta','Mexican'),
('Sushi World','Japanese'),
('Punjabi Tadka','Punjabi'),
('Cafe Aroma','Cafe'),
('BBQ Nation','Barbecue'),
('South Treat','South Indian');
```

```
INSERT INTO accounts(name,balance) VALUES
('Alice',5000),
('Bob',3000);
```

```
INSERT INTO orders (customer_id,restaurant_id,total,status,created_at) VALUES
(1,1,450.50,'Completed','2025-05-01 12:00:00'),
(1,3,320.00,'Completed','2025-05-05 14:00:00'),
(2,2,280.00,'Pending','2025-05-02 13:00:00'),
(2,5,600.00,'Completed','2025-05-08 18:00:00'),
(3,4,390.00,'Cancelled','2025-05-03 16:00:00'),
(3,1,250.00,'Completed','2025-05-10 19:00:00'),
(4,7,550.00,'Completed','2025-05-04 20:00:00'),
(4,3,450.00,'Pending','2025-05-09 11:30:00'),
(5,8,700.00,'Completed','2025-05-06 17:00:00'),
(5,2,260.00,'Completed','2025-05-12 18:20:00'),
(6,6,800.00,'Completed','2025-05-07 13:30:00'),
(6,9,950.00,'Pending','2025-05-15 20:30:00'),
(7,10,350.00,'Completed','2025-05-08 09:20:00'),
(7,5,420.00,'Completed','2025-05-18 12:15:00'),
(8,4,500.00,'Cancelled','2025-05-09 15:10:00'),
(8,1,620.00,'Completed','2025-05-20 18:45:00'),
(9,3,710.00,'Pending','2025-05-11 11:00:00'),
(9,8,380.00,'Completed','2025-05-22 17:30:00'),
(10,2,450.00,'Completed','2025-05-12 19:20:00'),
(10,6,900.00,'Completed','2025-05-24 21:00:00'),
(11,7,520.00,'Pending','2025-05-13 10:00:00'),
(11,4,610.00,'Completed','2025-05-26 13:00:00'),
(12,5,450.00,'Completed','2025-05-14 14:00:00'),
(12,8,780.00,'Completed','2025-05-28 18:00:00'),
(13,9,840.00,'Pending','2025-05-15 16:00:00'),
```

(13,1,370.00,'Completed','2025-05-29 20:00:00'),
(14,2,490.00,'Completed','2025-05-16 12:00:00'),
(14,6,860.00,'Cancelled','2025-05-30 19:00:00'),
(15,3,540.00,'Completed','2025-05-17 15:00:00'),
(15,10,310.00,'Completed','2025-06-01 18:00:00'),
(16,7,650.00,'Pending','2025-05-18 17:00:00'),
(16,5,290.00,'Completed','2025-06-02 20:00:00'),
(17,8,720.00,'Completed','2025-05-19 11:00:00'),
(17,9,900.00,'Completed','2025-06-03 13:00:00'),
(18,1,470.00,'Completed','2025-05-20 14:30:00'),
(18,2,350.00,'Pending','2025-06-04 19:30:00'),
(19,4,510.00,'Completed','2025-05-21 10:15:00'),
(19,6,680.00,'Completed','2025-06-05 16:30:00'),
(20,3,450.00,'Completed','2025-05-22 12:30:00'),
(20,7,760.00,'Cancelled','2025-06-06 18:30:00'),
(1,5,550.00,'Completed','2025-06-07 12:00:00'),
(2,8,620.00,'Completed','2025-06-08 14:00:00'),
(3,9,710.00,'Pending','2025-06-09 16:00:00'),
(4,10,450.00,'Completed','2025-06-10 18:00:00'),
(5,6,830.00,'Completed','2025-06-11 20:00:00'),
(6,2,310.00,'Completed','2025-06-12 13:00:00'),
(7,4,480.00,'Completed','2025-06-13 15:00:00'),
(8,7,690.00,'Pending','2025-06-14 17:00:00'),
(9,1,520.00,'Completed','2025-06-15 19:00:00'),
(10,5,410.00,'Completed','2025-06-16 21:00:00');

PART 2

TASK 1 Transactions (ACID)

a) Write a transaction that transfers ₹200 from one account to the other, but ends in ROLLBACK. Confirm both balances are unchanged afterward.

SQL Script:

```
116 -- ROLLBACK
117 • START TRANSACTION;
118
119 • UPDATE accounts
120   SET balance = balance - 200
121   WHERE name='Alice';
122
123 • UPDATE accounts
124   SET balance = balance + 200
125   WHERE name='Bob';
126
127 • ROLLBACK;
128 • SELECT * FROM accounts;
```

Result Grid:

id	name	balance
1	Alice	5000.00
2	Bob	3000.00

Output:

#	Time	Action	Message	Duration / Fetch
1	23:23:53	SELECT * FROM accounts LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec

b) Write the same transfer again, this time ending in COMMIT. Confirm both balances have changed.

SQL Script:

```
134 • START TRANSACTION;
135
136 • UPDATE accounts
137   SET balance = balance - 200
138   WHERE ID =1;
139
140 • UPDATE accounts
141   SET balance = balance + 200
142   WHERE ID =2;
143
144 • COMMIT;
145 -- -- COMMIT
146 • SELECT * FROM accounts;
```

Result Grid:

id	name	balance
1	Alice	4800.00
2	Bob	3200.00

Output:

#	Time	Action	Message	Duration / Fetch
19	23:42:54	UPDATE accounts SET balance = balance - 200 WHERE name='Alice'	Error Code: 1175. You are using safe update mode and you tried to update a table without a WHERE that uses...	0.000 sec
20	23:44:31	SELECT * FROM accounts LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
21	23:44:40	UPDATE accounts SET balance = balance - 200 WHERE ID =1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.016 sec
22	23:44:50	SELECT * FROM accounts LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
23	23:44:59	UPDATE accounts SET balance = balance + 200 WHERE ID =2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
24	23:45:03	SELECT * FROM accounts LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec

c) In 2–3 sentences, explain which ACID property guarantees the money is never "lost" in the middle of a transfer.

Atomicity is the ACID property that ensures a transaction is completed entirely or not at all. During a money transfer, both the debit and credit operations are treated as a single unit. If any part of the transaction fails, the entire transaction is rolled back, ensuring that no money is lost or partially transferred.

TASK 2 Joins

a) Write an INNER JOIN that lists every customer who has placed at least one order.

The screenshot shows MySQL Workbench with a query editor containing the following SQL code:

```
-- inner join
1 SELECT c.name, o.id, o.total
2 FROM customers c
3 INNER JOIN orders o
4 ON c.id=o.customer_id;
```

The Result Grid displays the following data:

name	id	total
Aarav Patel	1	450.50
Aarav Patel	2	320.00
Aarav Patel	41	550.00
Riya Shah	3	280.00
Riya Shah	4	600.00
Riya Shah	42	620.00
Vivaan Mehta	5	390.00
Vivaan Mehta	6	250.00
Vivaan Mehta	43	710.00
Ananya Joshi	7	550.00
Ananya Joshi	8	450.00
Ananya Joshi	44	450.00
Dhruv Patel	9	700.00
Dhruv Patel	10	260.00

The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
1	21:17:41	USE FoodDelivery	0 row(s) affected	0.000 sec
2	21:17:48	SELECT c.name, o.id, o.total FROM customers c INNER JOIN orders o ON c.id=o.customer_id LIMIT 0, 1000	50 row(s) returned	0.016 sec / 0.000 sec

b) Write a LEFT JOIN that lists every customer, including those with zero orders. Then add a WHERE clause to isolate only the customers who have never ordered.

The screenshot shows MySQL Workbench with a query editor containing the following SQL code:

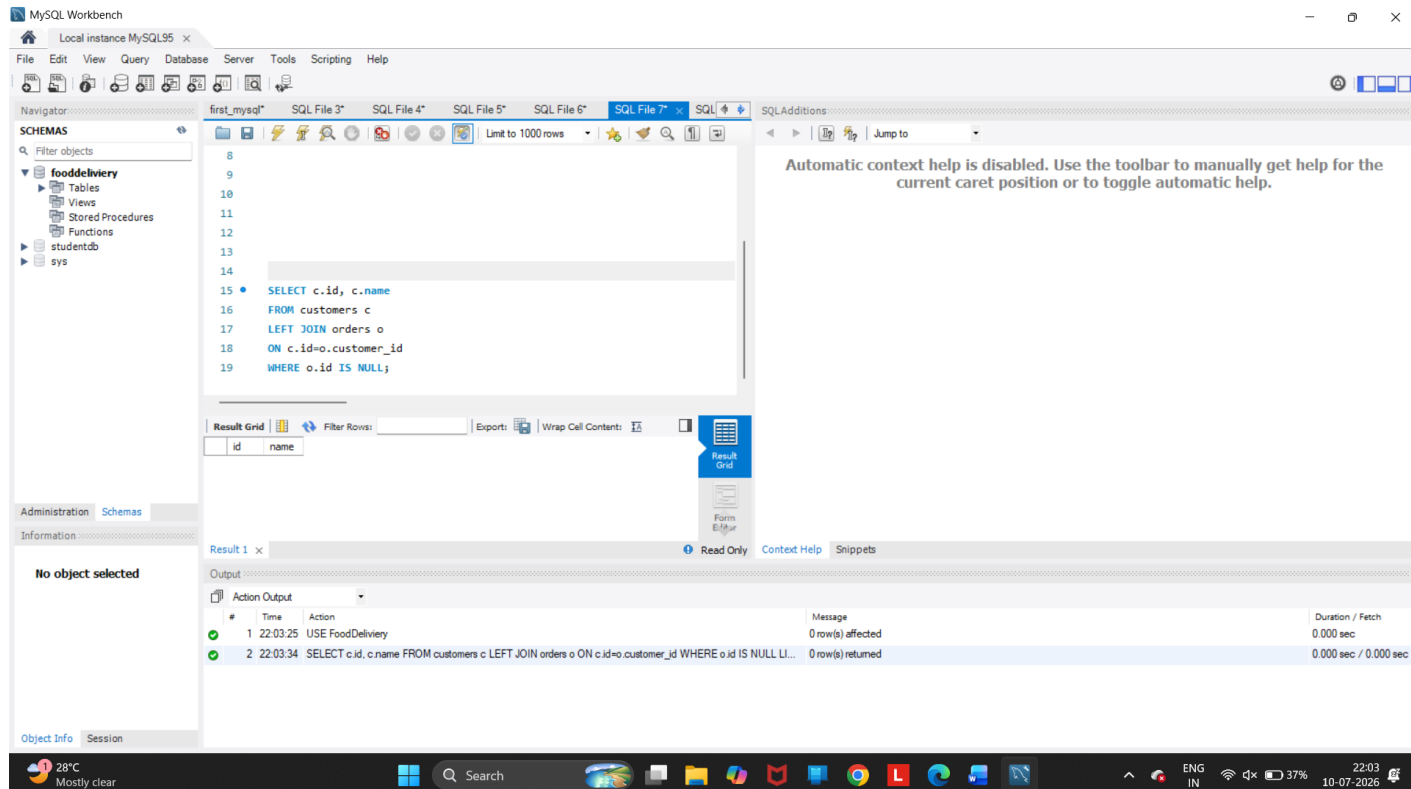
```
-- LEFT JOIN
7 Open a script file in this editor
8 o.id AS order_id
9 FROM customers c
10 LEFT JOIN orders o
11 ON c.id=o.customer_id
```

The Result Grid displays the following data:

id	name	order_id
1	Aarav Patel	1
1	Aarav Patel	2
1	Aarav Patel	41
2	Riya Shah	3
2	Riya Shah	4
2	Riya Shah	42
3	Vivaan Mehta	5
3	Vivaan Mehta	6
3	Vivaan Mehta	43
4	Ananya Joshi	7
4	Ananya Joshi	8
4	Ananya Joshi	44
5	Dhruv Patel	9
5	Dhruv Patel	10

The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
1	21:17:41	USE FoodDelivery	0 row(s) affected	0.000 sec
2	21:17:48	SELECT c.name, o.id, o.total FROM customers c INNER JOIN orders o ON c.id=o.customer_id LIMIT 0, 1000	50 row(s) returned	0.016 sec / 0.000 sec
3	21:18:55	SELECT c.id, c.name, o.id AS order_id FROM customers c LEFT JOIN orders o ON c.id=o.customer_id LIMIT 0, ...	50 row(s) returned	0.000 sec / 0.000 sec



c) In one sentence, explain why a marketing team would prefer the result of query (b) over query (a).

A marketing team would prefer the LEFT JOIN result because it includes customers who have never placed an order, allowing the company to target them with promotional offers, discounts, or reminder campaigns.

TASK 3 Indexing

- Pick a column that is not currently indexed (for example status or created_at).
- Run EXPLAIN ANALYZE on a query that filters by that column, and record the rows examined and actual time.

The screenshot shows a SQL IDE interface with the following components:

- SQL Editor:** Contains the query:

```
1 -- BEFORE INDEXING
2 EXPLAIN ANALYZE
3 SELECT *
4 FROM orders
5 WHERE status='Completed';
```
- SQLAdditions:** A sidebar on the right with a message: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help."
- Result Grid:** Displays the execution plan for the query. The first row is highlighted:

EXPLAIN
-> Filter: (orders.'status' = 'Completed') (cost=6 rows=5) (actual time=0.478..0.508 rows=36 loops=1)
- Output:** A table showing the execution details:

#	Time	Action	Message	Duration / Fetch
1	10:52:03	EXPLAIN ANALYZE SELECT * FROM orders WHERE status='Completed'	1 row(s) returned	0.000 sec / 0.000 sec

- Create an index on that column.

MySQL Workbench

Local instance MySQL95 x

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

fooddelivery

Tables

Views

Stored Procedures

Functions

studentdb

sys

SQL File 3* SQL File 4* SQL File 5* SQL File 6* SQL File 7* SQL File 8* SQL File 9* SQL File 10* SQL File 11* SQL File 12*

Limit to 1000 rows

```

1 CREATE INDEX idx_status
2 ON orders(status);
3 SHOW INDEX FROM orders;

```

Automatic context help is disabled. Use the toolbar to manually toggle help for the current caret position or to toggle automatic help.

Result Grid

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
orders	0	PRIMARY	1	id	A	50				BTREE		
orders	1	restaurant_id	1	restaurant_id	A	10			YES	BTREE		
orders	1	idx_status	1	status	A	3			YES	BTREE		
orders	1	idx_customer_date	1	customer_id	A	20			YES	BTREE		
orders	1	idx_customer_date	2	created_at	A	50			YES	BTREE		

Administration Schemas

Information

No object selected

Result 1 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
8	22:36:05	SELECT r.name AS Restaurant, o.total, o.status, o.created_at FROM orders o JOIN restaurants r ON o.restaurant_id = r.id	3 row(s) returned	0.000 sec / 0.000 sec
9	22:36:10	EXPLAIN EXPLAIN ANALYZE SELECT r.name AS Restaurant, o.total, o.status, o.created_at FROM orders o JOIN restaurants r ON o.restaurant_id = r.id	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version [10.0.0.0]	0.000 sec
10	22:36:19	EXPLAIN ANALYZE	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version [10.0.0.0]	0.000 sec
11	22:41:14	CREATE INDEX idx_status ON orders(status)	Error Code: 1061. Duplicate key name 'idx_status'	0.016 sec
12	22:41:41	CREATE INDEX idx_status ON orders(status)	Error Code: 1061. Duplicate key name 'idx_status'	0.000 sec
13	22:42:15	SHOW INDEX FROM orders	5 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

31°C Mostly clear

Search

ENG IN 18% 22:10:07:20

d) Re-run the exact same query and record the new numbers.

The screenshot displays the MySQL Workbench interface. The top pane shows a SQL script with the following content:

```

1  -- BEFORE INDEXING
2
3  -- AFTER INDEXING
4  • CREATE INDEX idx_status
5    ON orders(status);
6  • EXPLAIN ANALYZE
7    SELECT * FROM orders
8    WHERE status='Completed';
9

```

The bottom pane shows the execution results for the query. The first result is an EXPLAIN output:

```

EXPLAIN
-> Index lookup on orders using idx_status (status='Completed') (cost=4.35 rows=36) (actual time=0.0839..0.103 rows=36 loops=1)

```

The second result is an Action Output log showing the execution of the SQL script:

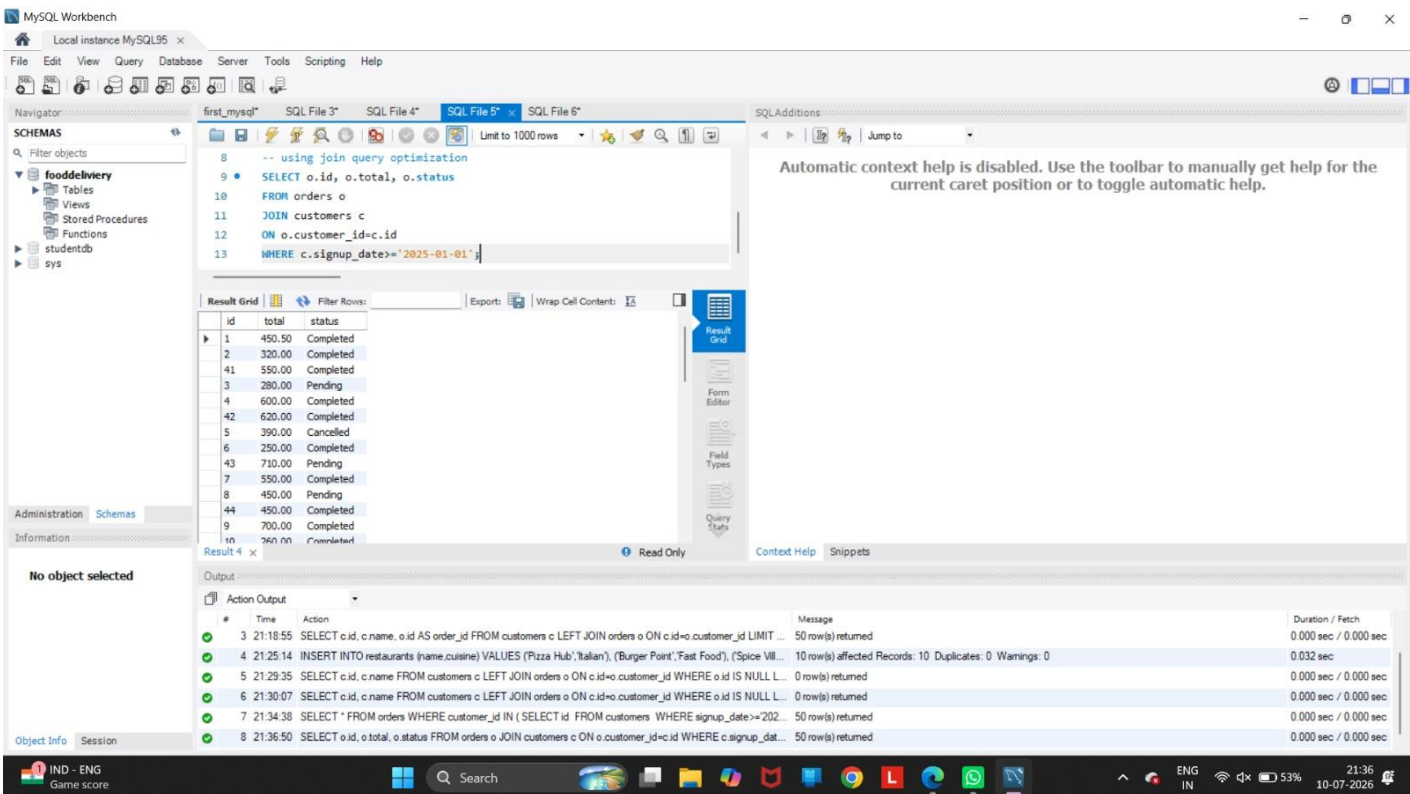
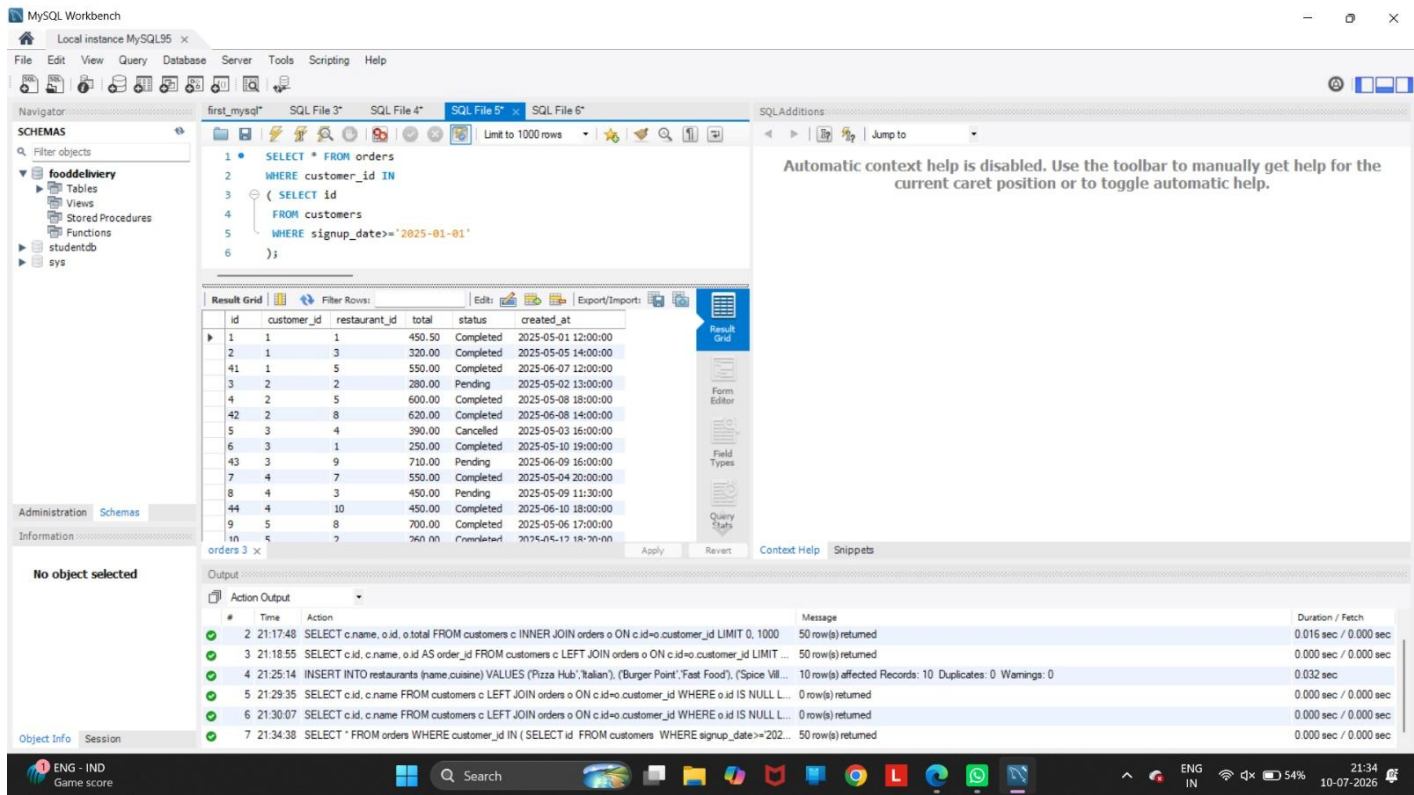
#	Time	Action	Message	Duration / Fetch
1	10:52:03	EXPLAIN ANALYZE SELECT * FROM orders WHERE status='Completed'	1 row(s) returned	0.000 sec / 0.000 sec
2	10:56:15	CREATE INDEX idx_status ON orders(status)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
3	10:56:15	EXPLAIN ANALYZE SELECT * FROM orders WHERE status='Completed'	1 row(s) returned	0.016 sec / 0.000 sec

e) Submit a one-line comparison of before vs. after.

After creating the index on the status column, MySQL used the `idx_status` index to locate matching rows more efficiently, reducing the amount of work required and improving query performance.

TASK 4 Query Optimization

a) Rewrite it as a JOIN instead of a subquery.



- b) Rewrite it again to select only the specific columns you'd actually need on a screen (avoid SELECT *).

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- EXPLAIN ANALYZE
SELECT * FROM orders
WHERE customer_id IN
( SELECT id
  FROM customers
  WHERE signup_date >= '2025-01-01'
);
```

The Result Grid shows the following data:

	id	customer_id	restaurant_id	total	status	created_at
1	1	1	1	450.50	Completed	2025-05-01 12:00:00
2	1	3	3	320.00	Completed	2025-05-05 14:00:00
41	1	5	5	550.00	Completed	2025-06-07 12:00:00
3	2	2	2	280.00	Pending	2025-05-02 13:00:00
4	2	5	5	600.00	Completed	2025-05-08 18:00:00
42	2	8	8	620.00	Completed	2025-06-08 14:00:00
5	3	4	4	390.00	Cancelled	2025-05-03 16:00:00
6	3	1	1	250.00	Completed	2025-05-10 19:00:00
43	3	9	9	710.00	Pending	2025-06-09 16:00:00
7	4	7	7	550.00	Completed	2025-05-04 20:00:00
8	4	3	3	450.00	Pending	2025-05-09 11:30:00
44	4	10	10	450.00	Completed	2025-06-10 18:00:00

The Output window shows the following actions:

#	Time	Action	Message	Duration / Fetch
4	21:25:14	INSERT INTO restaurants (name,cuisine) VALUES ('Pizza Hub','Italian'), ('Burger Point','Fast Food'), ('Spice Vill...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.032 sec
5	21:29:35	SELECT c.id, c.name FROM customers c LEFT JOIN orders o ON c.id=o.customer_id WHERE o.id IS NULL L...	0 row(s) returned	0.000 sec / 0.000 sec
6	21:30:07	SELECT c.id, c.name FROM customers c LEFT JOIN orders o ON c.id=o.customer_id WHERE o.id IS NULL L...	0 row(s) returned	0.000 sec / 0.000 sec
7	21:34:38	SELECT * FROM orders WHERE customer_id IN (SELECT id FROM customers WHERE signup_date >= '202...	50 row(s) returned	0.000 sec / 0.000 sec
8	21:36:50	SELECT o.id, o.total, o.status FROM orders o JOIN customers c ON o.customer_id=c.id WHERE c.signup_dat...	50 row(s) returned	0.000 sec / 0.000 sec
9	21:38:27	SELECT * FROM orders WHERE customer_id IN (SELECT id FROM customers WHERE signup_date >= '2025...	50 row(s) returned	0.000 sec / 0.000 sec

- c) Run EXPLAIN ANALYZE on your original query and your final rewritten version, and note what changed.

PART 3

TASK 5 The Customer Profile Page

a) Write a query that joins orders with restaurants (and customers, if needed) to return: restaurant name, order total, order status, and created_at — filtered to one specific customer, sorted by the most recent order first, limited to 5 results.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
1 SELECT r.name AS Restaurant,  
2 o.total, o.status, o.created_at  
3 FROM orders o  
4 JOIN restaurants r  
5 ON o.restaurant_id=r.id  
6 WHERE o.customer_id=5  
7 ORDER BY o.created_at DESC  
8 LIMIT 5;
```

The Results window displays the following data:

Restaurant	total	status	created_at
Sushi World	830.00	Completed	2025-06-11 20:00:00
Burger Point	260.00	Completed	2025-05-12 18:20:00
Cafe Aroma	700.00	Completed	2025-05-06 17:00:00

The Output window shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
6	21:30:07	SELECT c.id, c.name FROM customers c LEFT JOIN orders o ON c.id=o.customer_id WHERE o.id IS NULL L...	0 row(s) returned	0.000 sec / 0.000 sec
7	21:34:38	SELECT * FROM orders WHERE customer_id IN (SELECT id FROM customers WHERE signup_date>=202...	50 row(s) returned	0.000 sec / 0.000 sec
8	21:36:50	SELECT o.id, o.total, o.status FROM orders o JOIN customers c ON o.customer_id=c.id WHERE c.signup_d...	50 row(s) returned	0.000 sec / 0.000 sec
9	21:38:27	SELECT * FROM orders WHERE customer_id IN (SELECT id FROM customers WHERE signup_date>=2025...	50 row(s) returned	0.000 sec / 0.000 sec
10	21:40:11	SELECT o.id, o.total, o.status FROM orders o JOIN customers c ON o.customer_id=c.id WHERE c.signup_d...	50 row(s) returned	0.015 sec / 0.000 sec
11	21:43:18	SELECT r.name AS Restaurant, o.total, o.status, o.created_at FROM orders o JOIN restaurants r ON o.restaur...	3 row(s) returned	0.000 sec / 0.000 sec

b) Run EXPLAIN ANALYZE on it and record the rows examined and time taken.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
1 SELECT r.name, o.total, o.status, o.created_at
2 FROM orders o
3 JOIN restaurants r
4 ON o.restaurant_id=r.id
5 WHERE o.customer_id=5
6 ORDER BY o.created_at DESC
7 LIMIT 5;
```

The Results grid shows the following data:

name	total	status	created_at
Sushi World	830.00	Completed	2025-06-11 20:00:00
Burger Point	260.00	Completed	2025-05-12 18:20:00
Cafe Aroma	700.00	Completed	2025-05-06 17:00:00

The Output tab shows the execution plan and action output:

#	Time	Action	Message	Duration / Fetch
10	21:40:11	SELECT o.id, o.total, o.status FROM orders o JOIN customers c ON o.customer_id=c.id WHERE c.signup_dat...	50 row(s) returned	0.015 sec / 0.000 sec
11	21:43:18	SELECT r.name AS Restaurant, o.total, o.status, o.created_at FROM orders o JOIN restaurants r ON o.restaur...	3 row(s) returned	0.000 sec / 0.000 sec
12	21:44:27	SELECT r.name AS Restaurant, o.total, o.status, o.created_at FROM orders o JOIN restaurants r ON o.restaur...	3 row(s) returned	0.000 sec / 0.000 sec
13	21:46:06	CREATE INDEX idx_customer_date ON orders(customer_id,created_at)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
14	21:46:37	SELECT r.name AS Restaurant, o.total, o.status, o.created_at FROM orders o JOIN restaurants r ON o.restaur...	3 row(s) returned	0.000 sec / 0.000 sec
15	21:48:02	SELECT r.name, o.total, o.status, o.created_at FROM orders o JOIN restaurants r ON o.restaurant_id=r.id WH...	3 row(s) returned	0.000 sec / 0.000 sec

c) Create the index (or indexes) you think will make this query fast. Think about which column(s) appear in your WHERE clause and your ORDER BY.

MySQL Workbench

Local instance MySQL95

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

fooddelivery

Tables

Views

Stored Procedures

Functions

studentdb

sys

first_mysql* SQL File 3* SQL File 4* SQL File 5* SQL File 6* SQL File 7* SQL File 8* SQL File 9* SQL File 10* SQL File 11*

1 CREATE INDEX idx_customer_date
2 ON orders(customer_id,created_at);
3
4
5
6
7
8
9
10
11

Result Grid

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Vis
orders	0	PRIMARY	1	id	A	50				BTREE			YES
orders	1	restaurant_id	1	restaurant_id	A	10			YES	BTREE			YES
orders	1	idx_status	1	status	A	3			YES	BTREE			YES
orders	1	idx_customer_date	1	customer_id	A	20			YES	BTREE			YES
orders	1	idx_customer_date	2	created_at	A	50			YES	BTREE			YES

Administration Schemas

Information

No object selected

Result 1 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:03:25	USE FoodDelivery	0 row(s) affected	0.000 sec / 0.000 sec
2	22:03:34	SELECT c.id, c.name FROM customers c LEFT JOIN orders o ON c.id=o.customer_id WHERE o.id IS NULL LI...	0 row(s) returned	0.000 sec / 0.000 sec
3	22:25:48	EXPLAIN ANALYZE	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL serv...	0.000 sec
4	22:30:54	CREATE INDEX idx_customer_date ON orders(customer_id,created_at)	Error Code: 1061. Duplicate key name 'idx_customer_date'	0.000 sec
5	22:31:20	CREATE INDEX idx_customer_date ON orders(customer_id,created_at)	Error Code: 1061. Duplicate key name 'idx_customer_date'	0.000 sec
6	22:32:26	SHOW INDEX FROM orders	5 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

31°C Mostly clear

Search

ENG IN 23% 10-07-2026

d) Re-run the same query and compare the new numbers to step (b).

MySQL Workbench

Local instance MySQL95

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

fooddelivery

Tables

Views

Stored Procedures

Functions

studentdb

sys

SQL File 5* SQL File 6* SQL File 7* SQL File 8* SQL File 9* SQL File 10* SQL File 11*

2 ON orders(customer_id,created_at);
3
4 -- EXPLAIN ANALYZE
5
6 SELECT r.name AS Restaurant,
7 o.total, o.status, o.created_at
8 FROM orders o
9 JOIN restaurants r
10 ON o.restaurant_id=r.id
11 WHERE o.customer_id=5
12 ORDER BY o.created_at DESC
13 LIMIT 5;

Result Grid

Restaurant	total	status	created_at
Sushi World	830.00	Completed	2025-06-11 20:00:00
Burger Point	260.00	Completed	2025-05-12 18:20:00
Cafe Aroma	700.00	Completed	2025-05-06 17:00:00

Administration Schemas

Information

No object selected

Result 1 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
9	21:38:27	SELECT * FROM orders WHERE customer_id IN (SELECT id FROM customers WHERE signup_date>=2025...	50 row(s) returned	0.000 sec / 0.000 sec
10	21:40:11	SELECT o.id, o.total, o.status FROM orders o JOIN customers c ON o.customer_id=c.id WHERE c.signup_dat...	50 row(s) returned	0.015 sec / 0.000 sec
11	21:43:18	SELECT r.name AS Restaurant, o.total, o.status, o.created_at FROM orders o JOIN restaurants r ON o.restau...	3 row(s) returned	0.000 sec / 0.000 sec
12	21:44:27	SELECT r.name AS Restaurant, o.total, o.status, o.created_at FROM orders o JOIN restaurants r ON o.restau...	3 row(s) returned	0.000 sec / 0.000 sec
13	21:46:06	CREATE INDEX idx_customer_date ON orders(customer_id,created_at)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
14	21:46:37	SELECT r.name AS Restaurant, o.total, o.status, o.created_at FROM orders o JOIN restaurants r ON o.restau...	3 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

31°C Mostly clear

Search

ENG IN 47% 21:46 10-07-2026

e) Rewrite the query to select only the exact columns the profile page needs — no SELECT *.

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```

1 SELECT r.name, o.total, o.status, o.created_at
2 FROM orders o
3 JOIN restaurants r
4 ON o.restaurant_id=r.id
5 WHERE o.customer_id=5
6 ORDER BY o.created_at DESC
7 LIMIT 5;

```

The Results Grid shows the following data:

name	total	status	created_at
Sushi World	830.00	Completed	2025-06-11 20:00:00
Burger Point	260.00	Completed	2025-05-12 18:20:00
Cafe Aroma	700.00	Completed	2025-05-06 17:00:00

The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
10	21:40:11	SELECT o.id, o.total, o.status FROM orders o JOIN customers c ON o.customer_id=c.id WHERE c.signup_dat...	50 row(s) returned	0.015 sec / 0.000 sec
11	21:43:18	SELECT r.name AS Restaurant, o.total, o.status, o.created_at FROM orders o JOIN restaurants r ON o.restaur...	3 row(s) returned	0.000 sec / 0.000 sec
12	21:44:27	SELECT r.name AS Restaurant, o.total, o.status, o.created_at FROM orders o JOIN restaurants r ON o.restaur...	3 row(s) returned	0.000 sec / 0.000 sec
13	21:46:06	CREATE INDEX idx_customer_date ON orders(customer_id,created_at)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
14	21:46:37	SELECT r.name AS Restaurant, o.total, o.status, o.created_at FROM orders o JOIN restaurants r ON o.restaur...	3 row(s) returned	0.000 sec / 0.000 sec
15	21:48:02	SELECT r.name, o.total, o.status, o.created_at FROM orders o JOIN restaurants r ON o.restaurant_id=r.id WH...	3 row(s) returned	0.000 sec / 0.000 sec

f) Submit your before/after comparison, plus 2–3 sentences explaining why the index you chose helped.

Before indexing, MySQL scanned many rows to find the customer's orders and sort them by date. After creating a composite index on (customer_id, created_at), MySQL could quickly locate the matching rows in the required order, reducing both the number of rows examined and the execution time.